
Name (print):

Signature:

Instructions: This is a closed-book exam. Communicate your ideas *clearly* and *succinctly*. Write your solutions directly *and only* on this booklet. You may use other sheets of paper as scratch, but *do not submit anything other than this booklet*, as nothing else will be considered for grading. You may use either a pen or a pencil.

On problem	you got	out of
1		30
2		30
3		30
4		30
Total		120

(You may use this page to write your solution.)

► **Exercise 1.** In a binary search tree, let $h(x)$ be the height of a node x , such that $h(x) = 0$ (30) when x is the root, $h(x) = 1$ when x is a child of the root node, $h(x) = 2$ when x is a child of a node at height 1, and so on. Write an algorithm MAX-LEAF-HEIGHT-DIFF(t) that, given a non-empty binary search tree rooted at node t , computes the maximal difference between the height of any two *leaf* nodes in t . If there is only one leaf node, then the result is zero. A leaf is a node without child nodes. Also, analyze the complexity of your solution.

▷ *Solution 1*

In essence, MAX-LEAF-HEIGHT-DIFF(t) computes the difference between the *maximal* and *minimal* heights of any leaf node. We can compute the maximal and minimal leaf heights, and then return the difference. One way to do that is to compute the heights separately using two recursive functions (see code below on the left-hand side). Another way to do the same is to walk through the tree non-recursively, maintaining the current node height, and at each leaf node updating the maximal and minimal height (see the code on the right-hand side).

MAX-LEAF-HEIGHT-DIFF(t)

```
1  return MAX-LH( $t$ ) – MIN-LH( $t$ )
```

MAX-LH(t)

```
1  if  $t.left == \text{NIL}$  and  $t.right == \text{NIL}$ 
2      return 0
3  if  $t.left == \text{NIL}$ 
4      return 1 + MAX-LH( $t.right$ )
5  if  $t.right == \text{NIL}$ 
6      return 1 + MAX-LH( $t.left$ )
7  return 1 +  $\max(\text{MAX-LH}(t.left),$ 
                   $\text{MAX-LH}(t.right))$ 
```

MIN-LH(t)

```
1  if  $t.left == \text{NIL}$  and  $t.right == \text{NIL}$ 
2      return 0
3  if  $t.left == \text{NIL}$ 
4      return 1 + MIN-LH( $t.right$ )
5  if  $t.right == \text{NIL}$ 
6      return 1 + MIN-LH( $t.left$ )
7  return 1 +  $\min(\text{MIN-LH}(t.left),$ 
                   $\text{MIN-LH}(t.right))$ 
```

MAX-LEAF-HEIGHT-DIFF(t)

```
1   $min = \text{NIL}$ 
2   $max = \text{NIL}$ 
3   $prev = \text{NIL}$ 
4   $h = 0$ 
5  while  $t \neq \text{NIL}$ 
6      if  $prev == t.parent$  and  $t.left \neq \text{NIL}$ 
7           $prev = t$ 
8           $t = t.left$ 
9           $h = h + 1$ 
10     elseif  $prev \neq t.right$  and  $t.right \neq \text{NIL}$ 
11          $prev = t$ 
12          $t = t.right$ 
13          $h = h + 1$ 
14     else if  $t.left == \text{NIL}$  and  $t.left == \text{NIL}$ 
15         if  $min == \text{NIL}$  or  $h < min$ 
16              $min = h$ 
17         if  $max == \text{NIL}$  or  $h > max$ 
18              $max = h$ 
19      $prev = t$ 
20      $t = t.parent$ 
21      $h = h - 1$ 
22 return  $max - min$ 
```

Both solutions amount to one or two walks through the tree. So, in both cases, the complexity is $\Theta(n)$.

► **Exercise 2.** Consider the following algorithm $\text{ALGO-X}(A, B)$ that takes two arrays of numbers of total size $n = A.length + B.length$.

$\text{ALGO-X}(A, B)$

```
1   $x = 0$ 
2  for  $i = 1$  to  $A.length$ 
3      for  $j = i + 1$  to  $A.length$ 
4          if  $\text{CHECK-CONDITION}(A, i, j, B)$ 
5              if  $x < j - i$ 
6                   $x = j - i$ 
7  return  $x$ 
```

$\text{CHECK-CONDITION}(A, i, j, B)$

```
1  for  $p = i$  to  $j$ 
2      for  $q = 1$  to  $B.length$ 
3          if  $A[p] \leq B[q]$ 
4              return FALSE
5  return TRUE
```

Question 1: Briefly explain what ALGO-X does. Do not simply paraphrase the code. Instead, (10) explain the high-level semantics of ALGO-X independent of the code. Also, analyze the complexity of ALGO-X .

▷ *Solution 2.1*

ALGO-X returns the maximal length of any subsequence of A that contains values greater than all the values in B . The complexity is $\Theta(n^4)$.

Question 2: Write an algorithm LINEAR-ALGO-X that does exactly the same thing as ALGO-X (20) but with a complexity of $O(n)$. You must specify every algorithm you use.

▷ *Solution 2.2*

Since we are looking for sequences within A of numbers greater than *all* the values in B , we can first compute the maximal value in B and then use that value to check the elements in A , without having to scan B every time. With that, we simply check which elements in A are greater than that maximal value in B , and keep track of the longest sequence of those elements in A

LINEAR-ALGO-X(A, B)

```
1  if  $B.length == 0$ 
2      return  $A.length$ 
3   $m = B[1]$ 
4  for  $i = 2$  to  $B.length$ 
5      if  $m < B[i]$ 
6           $m = B[i]$ 
7   $r = 0$ 
8   $\ell = 0$ 
9  for  $i = 1$  to  $A.length$ 
10     if  $m < A[i]$ 
11          $\ell = \ell + 1$ 
12         if  $r < \ell$ 
13              $r = \ell$ 
14     else  $\ell = 0$ 
15  return  $r$ 
```

► **Exercise 3.** Consider the following algorithm $\text{ALGO-Y}(A, x)$ that takes an array of n numbers and a number x .

```
ALGO-Y( $A, x$ )
1   $B = \text{empty array}$ 
2   $C = \text{empty array}$ 
3   $D = \text{empty array}$ 
4  for  $i = 1$  to  $A.length$ 
5      if  $A[i] < x$ 
6          append  $A[i]$  to  $B$ 
7      elseif  $A[i] > x$ 
8          append  $A[i]$  to  $D$ 
9      else append  $A[i]$  to  $C$ 
10  $i = 1$ 
11 for  $j = 1$  to  $B.length$ 
12      $A[i] = B[j]$ 
13      $i = i + 1$ 
14 for  $j = 1$  to  $C.length$ 
15      $A[i] = C[j]$ 
16      $i = i + 1$ 
17 for  $j = 1$  to  $D.length$ 
18      $A[i] = D[j]$ 
19      $i = i + 1$ 
20 return ( $B.length + 1, B.length + C.length + 1$ )
```

Question 1: Briefly explain what ALGO-Y does. Do not simply paraphrase the code. Instead, (10) explain the high-level semantics of ALGO-Y independent of the code. Also, analyze the complexity of ALGO-Y.

▷ *Solution 3.1*

ALGO-Y partitions the input array A in three parts, from left to right, containing all the values originally in A that are less than, equal to, and greater than x , respectively. One or two of the three partitions may be empty. The complexity is $O(n)$. The return value is a pair of indexes (i, j) where i is the first position of the second partition and j is the first position of the third partition.

Question 2: Write an algorithm $\text{BETTER-ALGO-Y}(A, x)$ that does exactly what $\text{ALGO-Y}(A, x)$ (20) does, but *in-place*, meaning without using additional arrays. BETTER-ALGO-Y must run in $O(n)$ time.

▷ *Solution 3.2*

We can use two passes of the same in-place partitioning algorithm we use in quick sort.

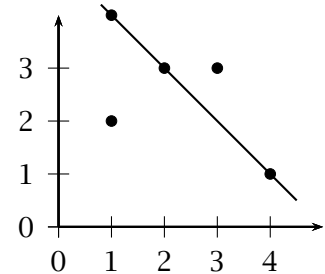
$\text{BETTER-ALGO-Y}(A, x)$

```
1   $p = 1$ 
2  for  $i = 1$  to  $A.length$ 
3      if  $A[i] < x$ 
4          swap  $A[i] \leftrightarrow A[p]$ 
5           $p = p + 1$ 
6   $q = p$ 
7  for  $i = p$  to  $A.length$ 
8      if  $A[i] == x$ 
9          swap  $A[i] \leftrightarrow A[q]$ 
10      $q = q + 1$ 
11  return  $(p, q)$ 
```

► **Exercise 4.** Write an algorithm MAXIMAL-DIAGONAL-OCCUPANCY(X, Y) that takes two arrays X and Y , each containing n numbers and representing the Cartesian coordinates of n points in the plane, respectively. Thus the input consists of n points with coordinates $(X[1], Y[1]), (X[2], Y[2]), \dots, (X[n], Y[n])$. MAXIMAL-DIAGONAL-OCCUPANCY must return the maximal number of points that are on the same ± 45 -degree diagonal. The complexity of MAXIMAL-DIAGONAL-OCCUPANCY must be strictly less than quadratic, so $o(n^2)$. (30)

For example, given the points $(1, 4), (1, 2), (2, 3), (3, 3), (4, 1)$, the result is 3, since the points $(1, 4), (2, 3), (4, 1)$ are on the same -45 -degrees diagonal, and there is no other diagonal line with more than 3 points on it. Notice that $(1, 2)$ and $(2, 3)$ are on the same $+45$ -degree diagonal.

Hint: Recall that a $+45$ -degree diagonal is a line defined by an equation $x - y = k$, for some constant k , and a -45 -degree diagonal is a line defined by an equation $x + y = k$.



► *Solution 4*

Points (x_1, y_1) and (x_2, y_2) are on the same 45 -degree diagonal if $x_1 - y_1 = x_2 - y_2$. Similarly, they are on the same -45 -degree diagonal if $x_1 + y_1 = x_2 + y_2$. In other words, the sum and difference of a point's coordinates identify the two diagonals the point is on, respectively. So, from the list of points, we can immediately get the two lists of diagonal identifiers, and then from those lists, we can get the most common value and its multiplicity. This last step can be done efficiently by sorting the list and then counting the adjacent identical identifiers.

MAXIMAL-DIAGONAL-OCCUPANCY(X, Y)

```

1   $n = X.length$ 
2   $D_1 =$  empty sequence
3   $D_2 =$  empty sequence
4  for  $i = 1$  to  $n$ 
5      append  $X[i] + Y[i]$  to  $D_1$ 
6      append  $X[i] - Y[i]$  to  $D_2$ 
7  sort  $D_1$ 
8  sort  $D_2$ 
9   $m = 0$ 
10 for  $i = 1$  to  $n$ 
11     if  $i > 1$  and  $D_1[i] == D_1[i - 1]$ 
12          $c = c + 1$ 
13         if  $c > m$ 
14              $m = c$ 
15     else  $c = 1$ 
16 for  $i = 1$  to  $n$ 
17     if  $i > 1$  and  $D_2[i] == D_2[i - 1]$ 
18          $c = c + 1$ 
19         if  $c > m$ 
20              $m = c$ 
21     else  $c = 1$ 
22 return  $m$ 
```